

— Mobile IoT Apps 2

Angular





— Angular

<https://angular.dev/>:

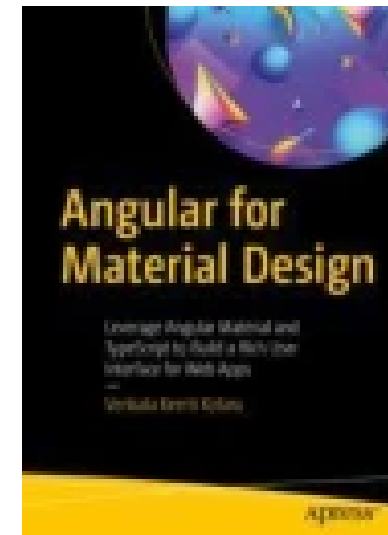
Angular is a web framework that empowers developers to build fast, reliable applications.

Oude Angular Website: <https://v17.angular.io/docs>

— Handboek: Angular for Material Design

Studiemateriaal - Theorie

Kotaru. (2020). Angular for Material Design:
Leverage Angular Material and TypeScript to Build a Rich User Interface for Web Apps
(1st ed. 2020. ed.). Berkeley, CA: Apress : Imprint: Apress.
ebook beschikbaar op <https://limo.libis.be/VIVES>
<https://doi-org.vives.e-bronnen.be/10.1007/978-1-4842-5434-9>
source code: <https://github.com/Apress/angular-for-material-design>



— Angular for Material Design

Inhoud

1. [Introduction](#)
2. [Getting Started](#)
3. [Modules](#)
4. [Angular: Components](#)
5. [Angular: Data Binding and Change Detection](#)
6. [Angular: Directives](#)
7. [Angular: Services and Dependency Injection](#)
8. [Material Design: User Input](#)
9. [Angular: Building Forms](#)
10. [Material Design: Additional Form Fields](#)
11. [Angular: Routing](#)
12. [Material Design: Navigation](#)
13. [Material Design: Layout](#)
14. [Material Design: Using Lists](#)
15. [Material Design: Alerts and Dialogs](#)
16. [Angular: HTTP Client](#)

— Angular for Material Design

1 - Introduction

Web technologies

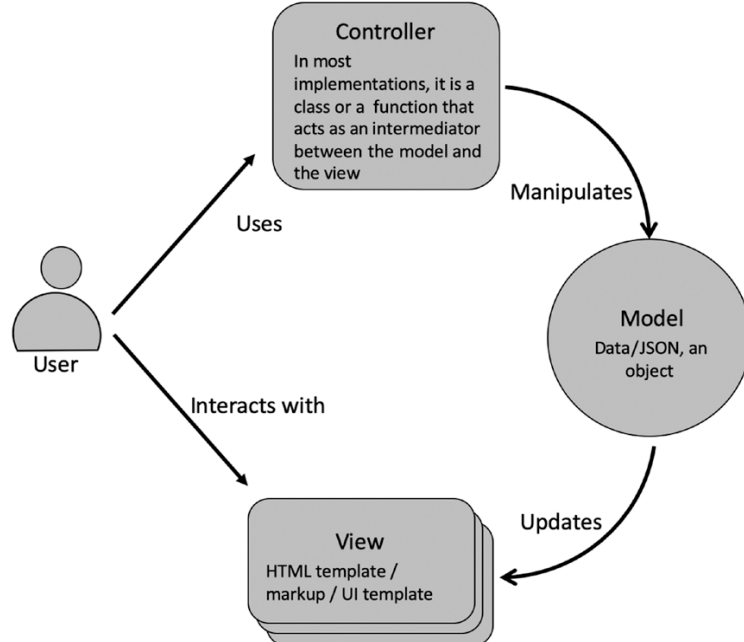
- Server => client
- standardize Javascript => ECMAScript
- Jaarlijks vernieuwd: ES9/ES2018, ...

— Angular for Material Design

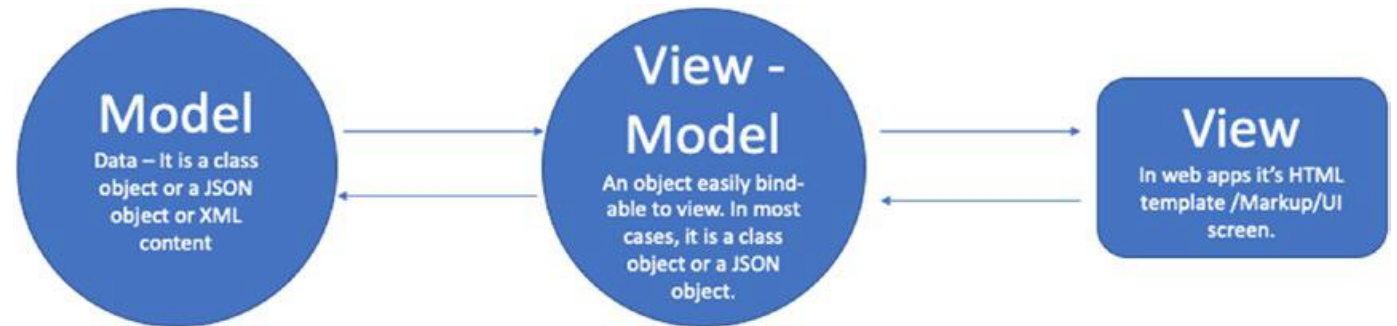
1 - Introduction

A brief history of Angular and its evolution

- Open source project van Google
- AngularJS 1.x => MV* framework (MVC of MVVM)



Model-View-Controller



Model-View-ViewModel

— Angular for Material Design

1 - Introduction

TypeScript

- superset of ECMAScript
- Datatypes, interfaces, generics en enums
- Backed by Microsoft

— Angular for Material Design

1 - Introduction

Material Design and the implementation of the Angular design language

- Google's design language
- Design language
 - nabootsen van real-world objecten => vandaar naam material
 - Kleurschema's, patronen, texturen, lettertypes
 - Consistente UI is belangrijk om de gebruiker gemakkelijker te laten navigeren door de App
- Angular Material Components: library met componenten die ontworpen zijn volgens Material Design Guidelines

— Angular for Material Design

2 – Getting Started

- Ontwerp van Hello World App met Typescript en Material Design
- Tools
 - Nodejs
 - Package manager:
 - npm of yarn
 - Installeren en beheren van libraries/apps met bijhorende dependencies
 - Angular CLI: `npm install --global @angular/cli`
 - Visual Studio Code
 - [Git for Windows](#)
 - [GitHub Desktop | Simple collaboration from your desktop](#)

— Intermezzo Angular

standalone components

- standalone components?
 - wijziging om complexiteit met modules te vermijden
 - imports dienen in de component te gebeuren:

```
@Component({  
  selector: 'app-book-list',  
  standalone: true,  
  imports: [CommonModule, MatCardModule],  
  templateUrl: './book-list.component.html',  
  styleUrls: ['./book-list.component.css']  
})
```

- klassieke modules kunnen met importProvidersFrom verkregen worden

```
export const appConfig: ApplicationConfig = {  
  providers: [provideRouter(routes), provideAnimations(), importProvidersFrom(HttpClientModule)],  
};
```

- Klassiek project met modules genereren: ng new --standalone=false
=> voeg "--standalone=false" toe als je handboek en dia's volgt
- Meer info op [\[EN\] Webinar: What's new in Angular 15? – YouTube](#)

— Angular for Material Design

2 – Getting Started

Getting Started with a Sample Application

- Option 1: Getting Started Using Angular CLI
 - `ng new superheroes --routing`
 - `cd superheroes`
 - `ng add @angular/material`
 - `npm start`
- Option 2: Material Design for an Existing Application Without Angular CLI
 - `npm install --save @angular/material @angular/cdk @angular/animations`
 - in `style.css`:
`@import "~@angular/material/prebuilt-themes/indigo-pink.css";`
 - in `index.html`:
`<link href="https://fonts.googleapis.com/css?family=Roboto:300,400,500" rel="stylesheet">`
`<link href="https://fonts.googleapis.com/icon?family=Material+Icons" rel="stylesheet">`

— Angular for Material Design

2 – Getting Started

Add Material Design Code Reference
voer ondertaande aanpassingen uit:

- Module Reference (p18, listing 2-1)
- Modify Template (p19, listing 2-2)
 - toolbar en card
- foutboodschap:
Module '"@angular/material"' has no exported member 'MatToolbarModule'.
=> oplossing:

```
// Import toolbar and card modules
import { MatToolbarModule } from '@angular/material/toolbar';
import { MatCardModule } from '@angular/material/card';
```

— Angular for Material Design

2 – Getting Started

- Sample application die wordt ontworpen:
 - An intuitive list of superheroes with images and interactive animations
 - The ability to filter
 - A detailed information screen for each superhero
 - A form to create a new superhero
 - The ability to edit a superhero
- [hans-naert/superheroes \(github.com\)](https://github.com/hans-naert/superheroes)
 - code les

— Angular for Material Design

2 – Getting Started

Files and Directories Generated by CLI (tekst in boek p21-p31)

- Configuration Files at the Root
 - Package.json
 - angular.json
 - tsconfig.js
 - tslint.json
- Default Application Directory Root: /src
 - TypeScript Configurations
 - index.html
 - main.ts

— Angular for Material Design

2 – Getting Started

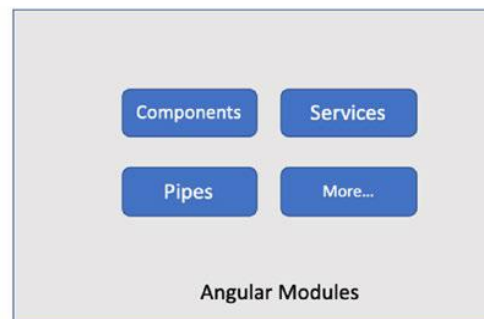
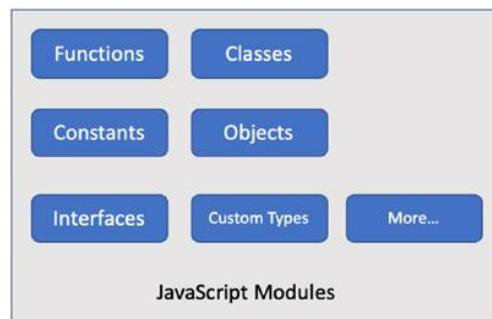
Files and Directories Generated by CLI (tekst in boek p21-p31)

- Application Directory: /src/app
 - Root Module: app.module.ts
 - App Component (Root Component)
 - Routing Module: app-routing.module.ts
- Scripts
 - Start script
 - Build script
 - Lint script

— Angular for Material Design

3 – Modules

- Modules => dependencies beheren
- Angular modules, zijn containers voor
 - componenten
 - directives
 - services
 - ...
- een andere module gebruiken met behulp van dependency injection



— Angular for Material Design

3 – Modules

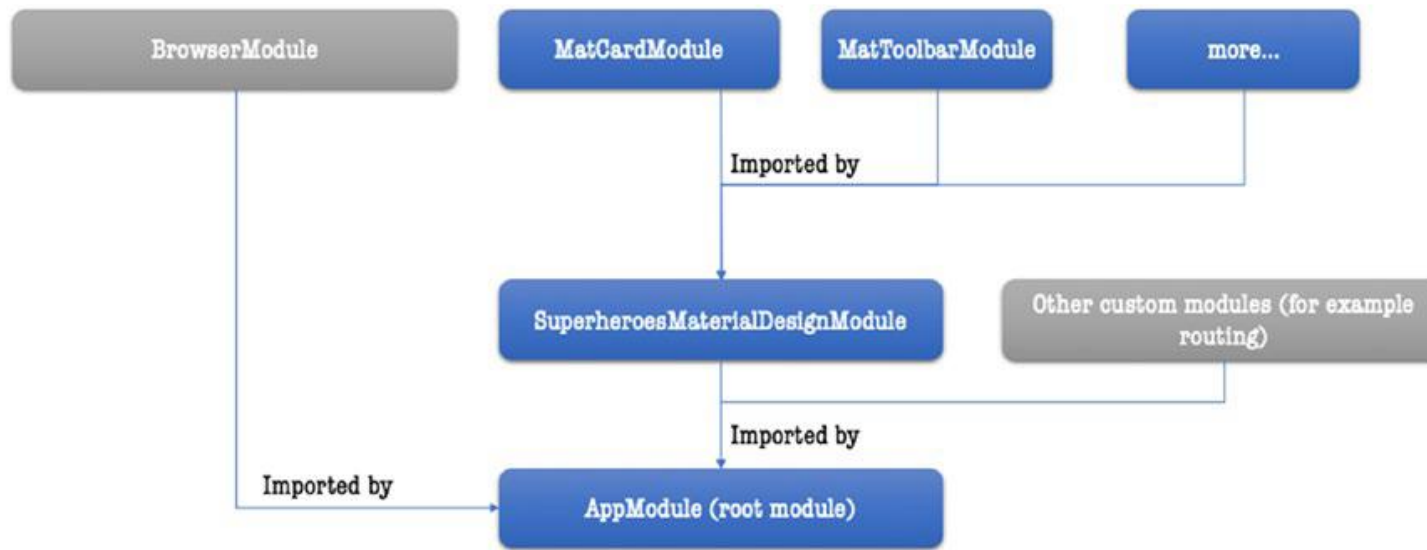
- Javascript modules: import / export
- Angular modules
 - @NgModule decorator: aanmaken van module
 - imports: modules die "required" zijn kan je hier importeren
 - declarations: Angular componenten en ... die deel uitmaken van de module
 - bootstrap: rootcomponent van een Angular application

```
@NgModule({  
  declarations: [  
    AppComponent  
  ],  
  imports: [BrowserModule],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

— Angular for Material Design

3 – Modules

Angular module aanmaken:
ng generate module superheroes-material-design



— Angular for Material Design

3 – Modules

- coding superheroes-material-design
 - listing 3-7 p39
 - listing 3-8 p40
 - listing 3-9 p40
- More About JavaScript Modules
 - The Primary Ways to Import and Export Code
 - Import All

— Angular for Material Design

4 - Angular: Components

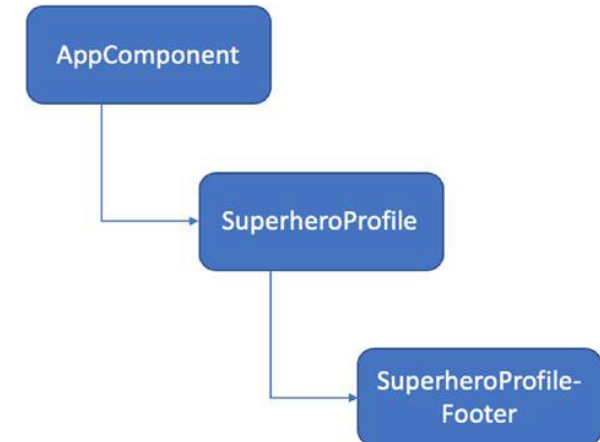
- components
 - maakt code herbruikbaar in een view
 - custom element - een custom DOM node
- Create a Component
 - superhero-toolbar:
ng generate component superheroes-material-design/superhero-toolbar
 - superhero-profile:
ng generate component superheroes-material-design/superhero-profile
listing 4-4 p52

— Angular for Material Design

4 - Angular: Components

Create a Component

- selector => tag name
- templateUrl => url van html code => neem listing 4-6 p53 over
- template => inline html
- styleUrls => url van css
- style => inline css
- encapsulation => css niet globaal
- providers => om een instantie van een klasse te maken en in de component te injecteren (dependency injection)
- viewProviders => do not provide projected content
- `<ng-content>` voor projecting content
=> zie listing 4-9 p58



— Angular for Material Design

4 - Angular: Components

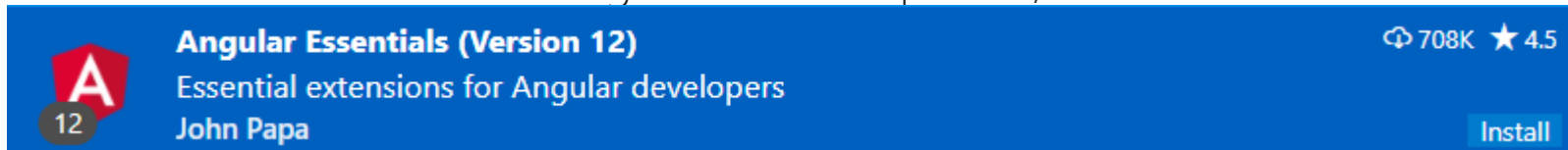
Create a Component

- providers <-> viewProviders:
 - maak superhero-text.ts (listing 4-11 p59)
 - gebruik in superhero-profile (listing 4-12 p60)
 - aangezien de footer projected is, werkt die enkel via "providers" en niet via "viewProviders"

— Angular for Material Design

4 - Angular: Components

- Input to a component
 - input voor een angular component => element attributes
 - @Input() directive
 - listing 4-15 en 4-16 p62
 - extensie om automatisch nodige modules importen, ...



- Validate Input with a Setter (listing 4-17)
- Validate Input with the ngOnChanges Lifecycle Hook (listing 4-18)



— Angular for Material Design

4 - Angular: Components

Output from a Component

- emitting events (listing 4-19 tot en met 4-22)



— Angular for Material Design

4 - Angular: Components

Lifecycle Hooks

- `ngOnInit()`
- `ngOnDestroy()`
- `ngOnChanges()`
- `ngDoCheck()`
- `ngAfterViewInit()` en `ngAfterViewChecked()`
- `ngAfterContentInit()` en `ngAfterContentChecked()`

— Intermezzo SCSS

Thema kleuren gebruiken in eigen CSS (1/3)

- Sass: [Sass: Sass Basics \(sass-lang.com\)](http://sass-lang.com)

SCSS Sass

```
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: 100% $font-stack;
  color: $primary-color;
}
```

CSS

```
body {
  font: 100% Helvetica, sans-serif;
  color: #333;
}
```

- Modules en Mixins

SCSS Sass

```
// _base.scss
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: 100% $font-stack;
  color: $primary-color;
}
```

```
// styles.scss
@use 'base';

.inverse {
  background-color: base.$primary-color;
  color: white;
}
```

SCSS Sass

```
@mixin theme($theme: DarkGray) {
  background: $theme;
  box-shadow: 0 0 1px rgba($theme, .25);
  color: #fff;
}

.info {
  @include theme;
}

.alert {
  @include theme($theme: DarkRed);
}

.success {
  @include theme($theme: DarkGreen);
}
```

— Intermezzo SCSS

Thema kleuren gebruiken in eigen CSS (2/3)

- Gebruik Sass in Angular project: extensie scss
- Thema zelf definiëren in Sass zodat variabelen op andere plaatsen bereikbaar zijn: (info op [Theming Angular Material | Angular Material](#))
- styles.scss:

```
@use '@angular/material' as mat;  
@include mat.core();  
$angular-app-scss-theme: mat.define-theme((  
  color: (  
    theme-type: light,  
    primary: mat.$azure-palette,  
    tertiary: mat.$blue-palette,  
  ),  
  density: (  
    scale: 0,  
  ),  
));
```

```
:root {  
  @include mat.all-component-  
themes($angular-app-scss-theme);  
}  
  
html, body { height: 100%; }  
body { margin: 0; font-family: Roboto,  
"Helvetica Neue", sans-serif; }
```

— Intermezzo SCSS

Thema kleuren gebruiken in eigen CSS (3/3)

- Gebruik thema kleuren in SCSS: [Theming Angular Material | Angular Material](#)

```
@use '@angular/material' as mat;  
@import 'styles.scss';  
  
p {  
  color: mat.get-theme-color($angular-app-scss-theme, on-primary);  
  background-color: mat.get-theme-color($angular-app-scss-theme, primary);  
}
```

- Voeg het pad naar styles.scss toe aan de build options

```
"stylePreprocessorOptions": {  
  "includePaths": [  
    "src"  
  ]  
}
```

- Codevoorbeeld: [hans-naert/angular-app-scss \(github.com\)](https://github.com/hans-naert/angular-app-scss)

— Angular for Material Design

5 - Angular: Data Binding and Change Detection

- Data binding
 - binds HTML with typescript
 - minder foutgevoelig dan via Javascript de DOM elementen lezen/aanpassen
- Interpolation
 - Angular templates in HTML => {{ ... }}
 - je kan variabelen gebruiken van de component
 - de view wijzigt automatisch als de variabelen wijzigen
 - listing 5-1

— Angular for Material Design

5 - Angular: Data Binding and Change Detection

- Property Binding [attribute]="typescript variable"
 - `<img [src]="imageUrl" />`
- Alternate Syntax for Property Binding
 - `<input type="text" bind-disabled="isTextDisabled" />`
- Class Binding (CSS classes)
 - listing 5-10 en 5-11
- ngClass Directive
 - listing 5-12 en 5-13

— Angular for Material Design

§ - Angular: Data Binding and Change Detection

Style Binding

- `<div [style.font-size.pt]="fontSize">`
listing 5.14-5.15
- `ngStyle`
`<div [ngStyle]="styles">{{name}}</div>`
listing 5.16-5.17

— Angular for Material Design

\$ - Angular: Data Binding and Change Detection

Event Binding

--contents in HTML template file---

```
<input type="text" (input)="handleEvent($event)" />
```

--TypeScript event handler---

```
handleEvent(event) {  
    console.log("$event object", event);  
    console.log(`event type`, event.type);  
    console.log("Value on the event target element",  
        event.target.value);  
}
```

— Angular for Material Design

§ - Angular: Data Binding and Change Detection

- Two-Way Data Binding
 - ngModel directive - out of FormsModule
listing 5.22
 - `<input type="text" [(ngModel)]="superheroName"/>`
listing 5.23 en 5.24
- Change Detection
 - Default Change Detection Strategy
uses "==="
 - OnPush Change Detection Strategy
compares references => use immutable objects

— Angular for Material Design

§ - Angular: Data Binding and Change Detection

- Create a form that allows the user to input the following information about a dinosaur.
 - Dinosaur name
 - Description
 - Family drop-down menu: the values are Herrerasauridae, Ceratopsidae, Tyrannosauridae, and Plateosauridae
 - Submit button
- The user should see the dinosaur information in a separate section of the page. The section could be to the side of or below the form. Use interpolation to show the dinosaur values in this section. Use ngModel and two-way data binding to persist the form's values in a model object. When the Submit button is clicked, the console logs the values provided by the user.

— Angular for Material Design

6 - Angular: Directives

- Directives
 - components: aanmaak custom elements
 - structural directives: customize/modify structure of a view
 - iterate over a list - show on a view
 - conditionally show an view (that is hidden by default)
 - attribute directives: bvb ngStyle

— Angular for Material Design

6 - Angular: Directives

- ng-template (een Angular element)
 - template variabele gebruiken:
`<div *ngTemplateOutlet="heroContactDetails"> </div>`
 - template variabele aanmaken:
`<ng-template #heroContactDetails>`
`<h3> ...</h3> ...`
`</ng-template>`

— Angular for Material Design

6 - Angular: Directives

- *ngIf (een structural directive)
 - gebruik *ngIf:
`<div *ngIf="showContacts; then contactDetails else hiddenText"> </div>`
 - checkbox
`<input class="specialInput" type="checkbox" [(ngModel)]="showContacts">`
Show contact details
 - template variabele toevoegen:
`<ng-template #hiddenText>`
`<hr />`
*Contact information is hidden.
`</ng-template>`

— Angular for Material Design

6 - Angular: Directives

- ng-container
 - om HTML elementen in een container te plaatsen
 - te gebruiken wanneer het gebruik van een <div> niet mogelijk is door bvb CSS die er zou op toegepast worden
 - gebruik ng-container:
`<ng-container *ngIf="showContacts; then contactDetails else hiddenText"> </ng-container>`

— Angular for Material Design

6 - Angular: Directives

- switch case
 - ngSwitch: variabele die moet vergeleken worden
 - ngSwitchCase: één case
 - ngSwitchDefault: default case
- listing 6-6 p101

— Angular for Material Design

6 - Angular: Directives

- *ngFor en ngForOf
 - shorthand:
`<li *ngFor="let hero of superheroes"> {{hero}} </li>`
 - expanded form with ngForOf:
`<ng-template ngFor let-hero [ngForOf]="superheroes">
 {{hero}}
</ng-template>`
 - local variables
 - index
 - first
 - last
 - even
 - odd
- ng generate component superheroes-material-design/home
Listing 6-7 en 6-10

— Angular for Material Design

6 - Angular: Directives

- ngStyle
 - `<div [ngStyle]="titleStyle">Chhotta Bheem</div>`
 - `get titleStyle(){
 return {
 'text-decoration': this.isStrikeThrough ? 'line-through' : 'none',
 'font-weight': this.isBold ? 'bold': 'normal',
 'font-style': this.isItalic ? 'italic': 'normal',
 'font-size': '24px'
 };
}`
 - `<input class="specialInput" type="checkbox" [(ngModel)]="isBold">`

— Angular for Material Design

6 - Angular: Directives

- Oefening
Create a list screen that shows a dinosaur's listing. Also iterate through the following dinosaur family list to show check boxes next to each item. When the user selects a family, show only the dinosaurs that belong to the selected family.
 - Abelisauridae
 - Noosauridae
 - Megalosauridae

— Angular for Material Design

7 - Angular: Services and Dependency Injection

Angular service

- heeft geen view
- code die in background werkt (in browser)

Komt ook aan bod: asynchronous constructs

- asynchronous function calls met callbacks
- Javascript Promises en observables
- RxJS voor observables implementations

— Angular for Material Design

7 - Angular: Services and Dependency Injection

asynchronous function calls

- synchronous: wachten op return resultaat
- function callback: na afloop wordt callback function uitgevoerd
`console.log("message 1");`
`setTimeout(function(){ console.log("callback invoked after 3 seconds") } , 3000);`
`console.log("message 2");`

— Angular for Material Design

7 - Angular: Services and Dependency Injection

JavaScript Promises

- function callbacks => code out of order
- with promises: function returns promise object, not the real value, it is a promise that the task will be attempted
- Promise returned by function call:
let promiseRef: Promise = aTimetakingTask(false);

- Promise constructor:
Promise(executorCallback(resolve, reject));

```
aTimetakingTask (input){  
  return new Promise( (resolve, reject) => {  
    if(input){  
      resolve("job well done");  
    }else{  
      reject("job failed");  
    }  
  })  
}
```

— Angular for Material Design

7 - Angular: Services and Dependency Injection

JavaScript Promises

- Promise reference used by the calling function

```
let promiseRef: Promise = this. aTimetakingTask (false);  
promiseRef.then(  
    (result) => console.log(result),  
    (error) => console.log(error)  
);
```

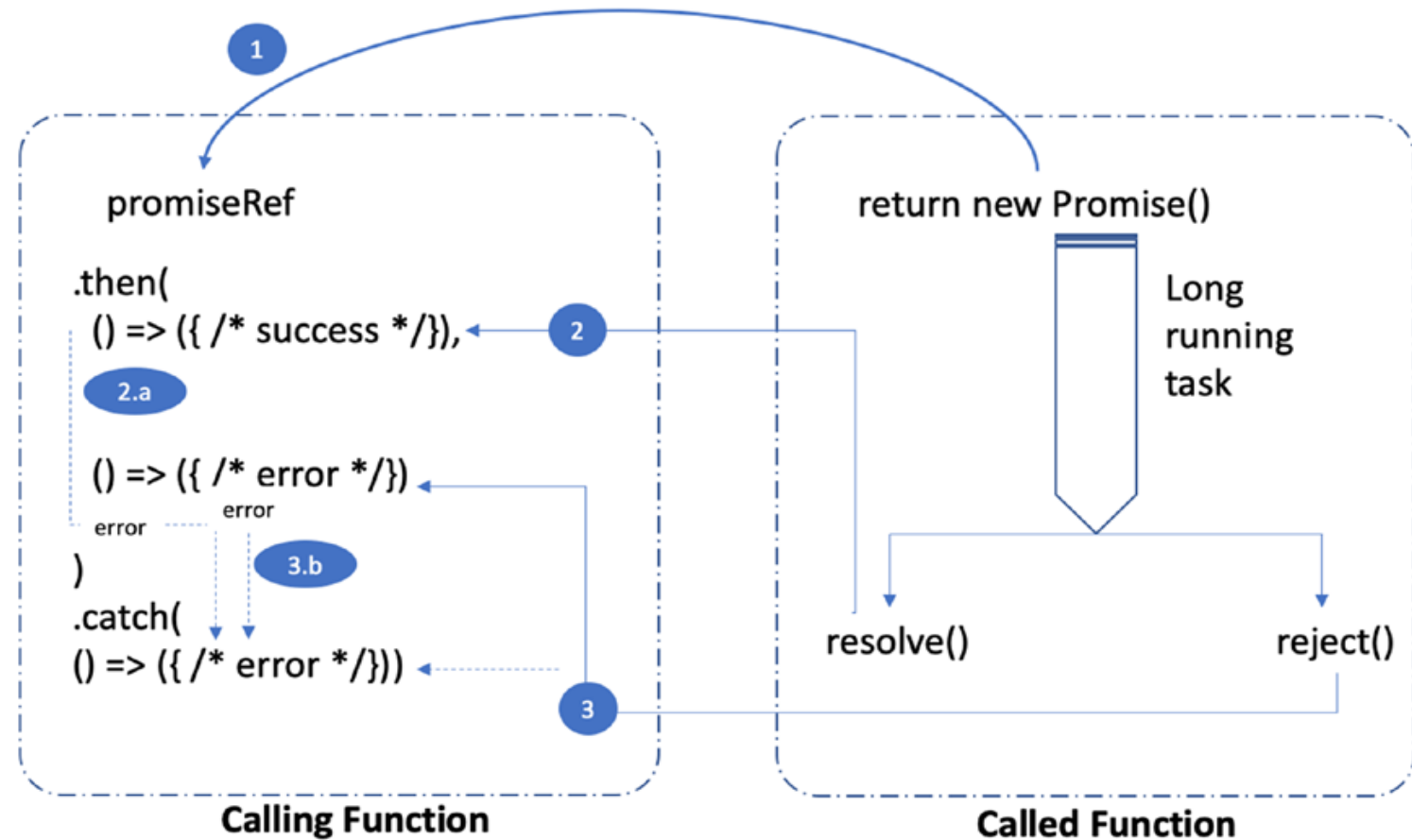
```
promiseRef  
    .then(  
        (result) => {  
            console.log(result)  
            /* More lines of code  
               making use of data returned by the promise. */  
        })  
    .catch(  
        (error) => console.log(error)  
    );
```

— Angular for Material Design

7 - Angular: Services and Dependency Injection

JavaScript Promises

- using promises



bron: [Angular for Material Design](#)

— Angular for Material Design

7 - Angular: Services and Dependency Injection

reactive programming with RxJS

- [How to Read an RxJS Marble Diagram - DEV Community](#)
- RxJS (Reactive Extensions for JavaScript)
- observable: provides a collection of values asynchronously as and when available
- RxJS implements an observable
- observable streams values to the subscriber
promise => 1 value / observable => stream of values
- use cases in Angular
 - remote service API (HTTP calls)
 - user prompts and dialog boxes

— Angular for Material Design

7 - Angular: Services and Dependency Injection

create an observable

```
import { Observable } from 'rxjs';
observableSample(input): Observable<string>{
  return new Observable((observer) => {
    let values = ["value1", "value2", "value3", "value4"];
    if(input){
      values.map( value => observer.next(value));
    }else{
      observer.error("error");
    }
    observer.complete();
  });
}
```

observable subscription

```
let observableRef: Observable<string> = this.observableSample(true);
observableRef.subscribe(
  (result) => console.log(result),
  (error) => console.log(error),
  () => console.log("complete")
)
```

— Angular for Material Design

Observables

- Reactive X (<http://reactivex.io/>) is een library voor asynchroon en event gebaseerd te programmeren
- Reactive X bestaat voor verschillende talen, RxJS is de versie voor javascript

Languages

- Java: [RxJava](#)
- JavaScript: [RxJS](#)
- C#: [Rx.NET](#)
- C#(Unity): [UniRx](#)
- Scala: [RxScala](#)
- Clojure: [RxClojure](#)
- C++: [RxCpp](#)
- Lua: [RxLua](#)
- Ruby: [Rx.rb](#)
- Python: [RxPY](#)
- Go: [RxGo](#)

- Groovy: [RxGroovy](#)

- JRuby: [RxJRuby](#)

- Kotlin: [RxKotlin](#)

- Swift: [RxSwift](#)

- PHP: [RxPHP](#)

- Elixir: [reaxive](#)

- Dart: [RxDart](#)

ReactiveX for platforms and frameworks

- [RxNetty](#)

- [RxAndroid](#)

- [RxCocoa](#)

— Angular for Material Design

Observables

Better codebases



Functional

Avoid intricate stateful programs, using clean input/output functions over observable streams.



Less is more

ReactiveX's operators often reduce what was once an elaborate challenge into a few lines of code.



Async error handling

Traditional try/catch is powerless for errors in asynchronous computations, but ReactiveX is equipped with proper mechanisms for handling errors.

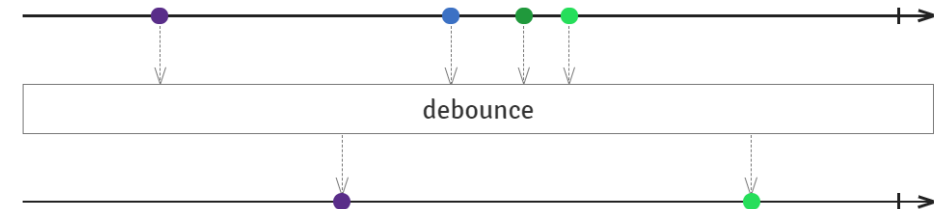


Concurrency made easy

Observables and Schedulers in ReactiveX allow the programmer to abstract away low-level threading, synchronization, and concurrency issues.

The Observer pattern done right

ReactiveX is a combination of the best ideas from the **Observer** pattern, the **Iterator** pattern, and **functional programming**



— Angular for Material Design

Observables

Observables maken het onderstaande raster compleet

	Single	Multiple
Pull	Function	Iterator
Push	Promise	Observable

- A **Function** is a lazily evaluated computation that **synchronously** returns a **single** value on invocation.
- A **generator** is a lazily evaluated computation that **synchronously** returns **zero to (potentially) infinite** values on iteration.
- A **Promise** is a computation that **may (or may not) eventually return** a **single** value.
- An **Observable** is a lazily evaluated computation that can synchronously or asynchronously **return zero to (potentially) infinite** values from the time it's invoked onwards.

— Angular for Material Design

Observables

[hans-naert/observables \(github.com\)](https://github.com/hans-naert/observables)

- Observables aanmaken – create

```
var observable = new Observable(function(observer) {  
    observer.next(1);  
    observer.next(2);  
    observer.next(3);  
    setTimeout(() => { observer.next(4); observer.complete(); }, 1000);  
});
```

- Observable gebruik - subscribe

```
console.log('just before subscribe');  
observable.subscribe({  
    next: x => console.log('got value ' + x),  
    error: err => console.error('something wrong occurred: ' + err),  
    complete: () => console.log('done'),  
});  
console.log('just after subscribe');
```

— Angular for Material Design

Observables

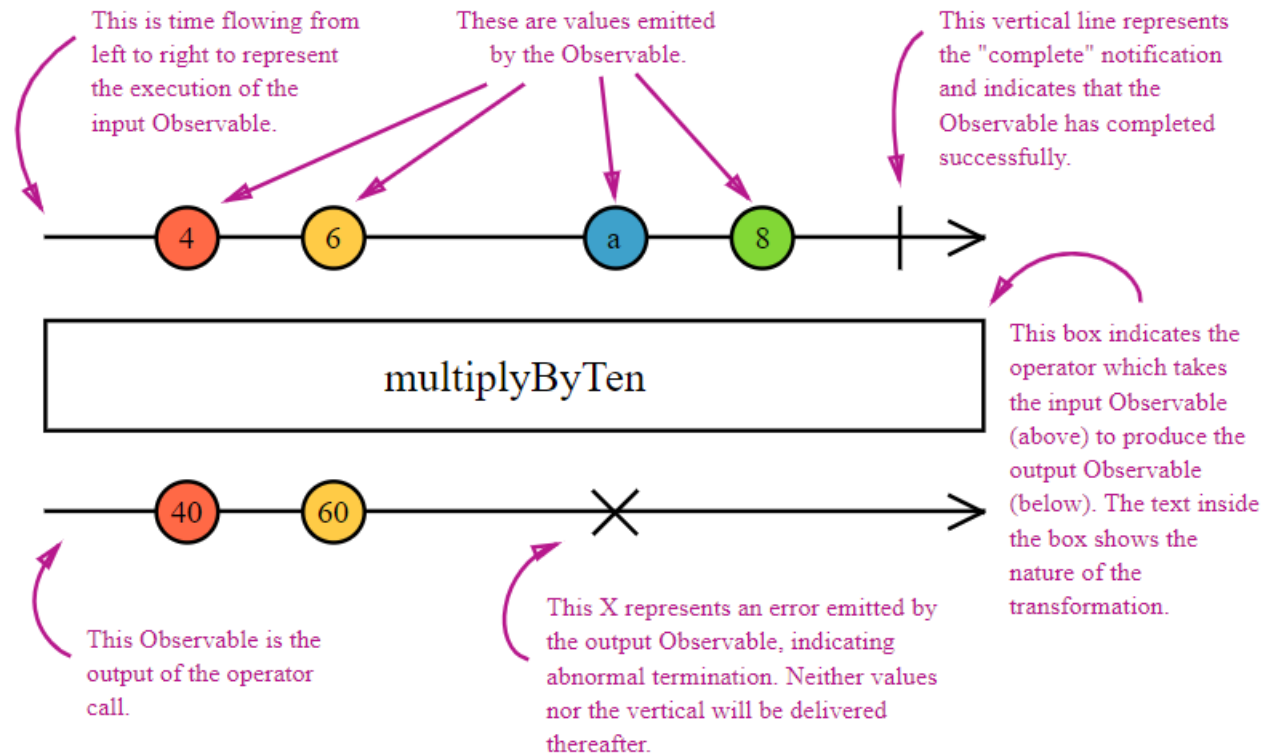
Resultaat

```
just before subscribe  
got value 1  
got value 2  
got value 3  
just after subscribe  
got value 4  
done
```

— Angular for Material Design

Observables

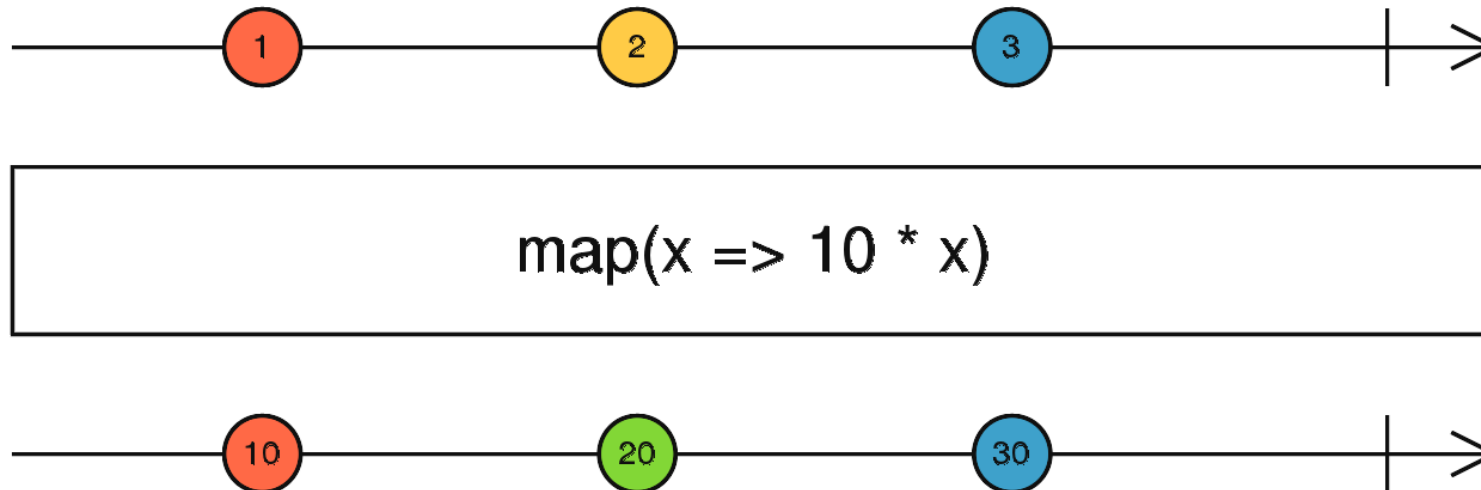
Marble diagrams om de werking van operatoren voor te stellen



— Angular for Material Design

Observables

map: <https://rxjs-dev.firebaseapp.com/api/operators/map>



— Angular for Material Design

7 - Angular: Services and Dependency Injection

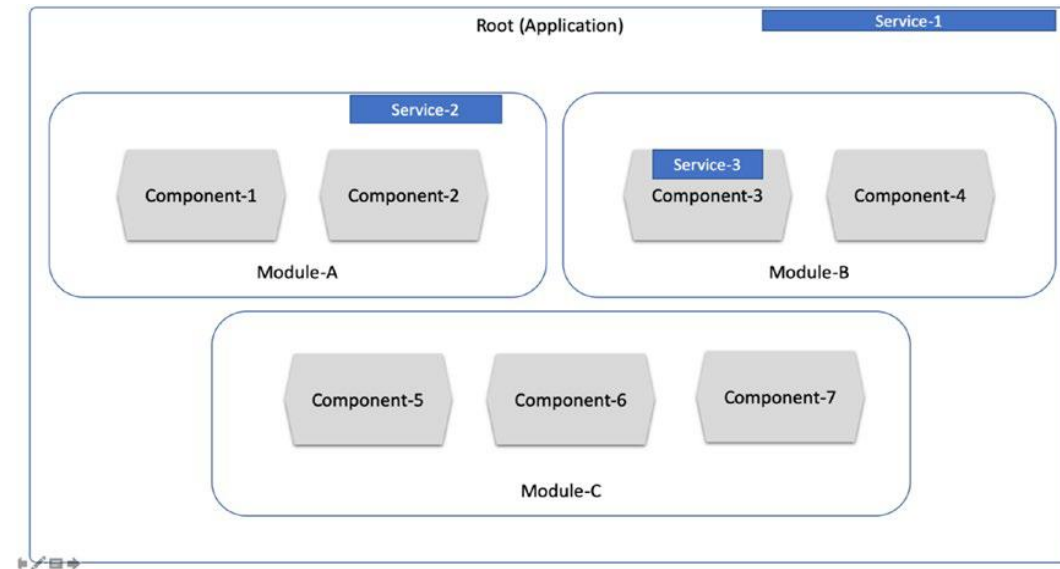
- Angular Services
 - data die over de hele App nodig is (bvb superhero data)
 - functie die data verzameld en transformeert - zodat views via databinding de huidige data kunnen tonen
- Create a service:
ng generate service data-services/superhero-data

— Angular for Material Design

7 - Angular: Services and Dependency Injection

Dependency injection in Angular

- Provider for a service
 - Angular creëert injectoren
 - scope:
 - Root
 - Module
 - component



bron: [Angular for Material Design](#)

— Angular for Material Design

Event.target.value

Cast naar HTMLInputElement nodig...

- Typescript:

```
getTargetValue(event: Event) {  
  return (event.target as HTMLInputElement).value
```

- Template code:

[Angular - Template type checking](#)

```
<input type="text" (input)="superheroName= $any($event.target).value;" />
```

— Angular for Material Design

7 - Angular: Services and Dependency Injection

Service - injected at root - neem code over van [angular-for-material-design/superhero-data.service.ts at master · Apress/angular-for-material-design \(github.com\)](https://github.com/Apress/angular-for-material-design/blob/master/src/app/services/superhero-data.service.ts) en voeg het superhero model toe

```
import { Injectable } from '@angular/core';
@Injectable({
  providedIn: 'root'
})
export class SuperheroDataService {
  getSuperheroes(): Array<Superhero> {
    return [
      {
        // Superhero objects go here.
        // removed code for brevity
      },
    ],
    constructor() {}
  }
}
```

— Angular for Material Design

7 - Angular: Services and Dependency Injection

- Service - module level:
in @NgModule({ providers :[SuperheroDataService,
- Service - component level:
in @Component({providers :[SuperheroDataService,
- Data van observable gebruiken:

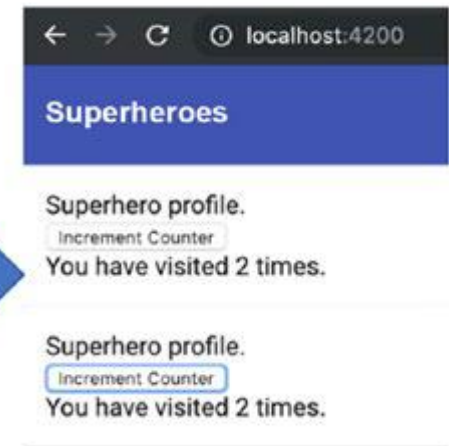
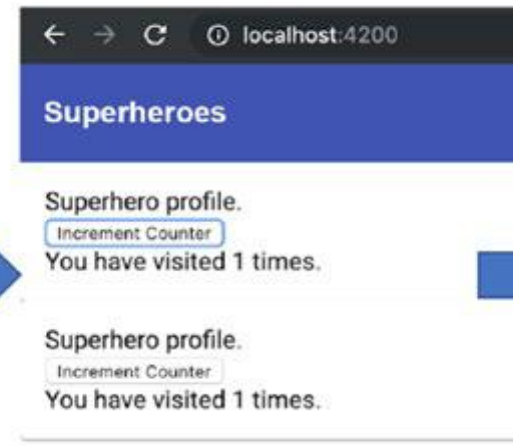
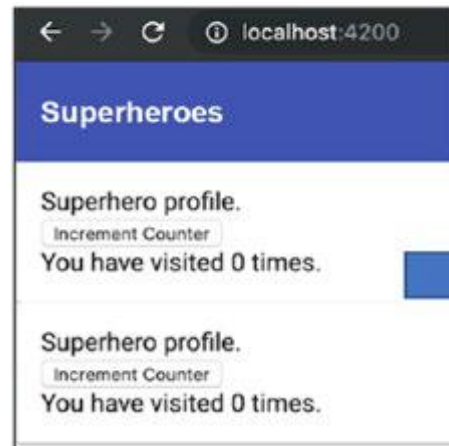
```
import { SuperheroDataService } from 'src/app/data-services/superhero-data.service';  
  
...  
constructor(private dataService: SuperheroDataService) { }  
  
ngOnInit(): void {  
    this.dataService.getSuperheroes().subscribe( data => console.log(data));  
}
```

— Angular for Material Design

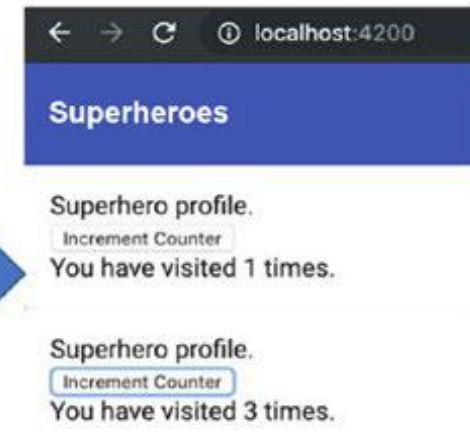
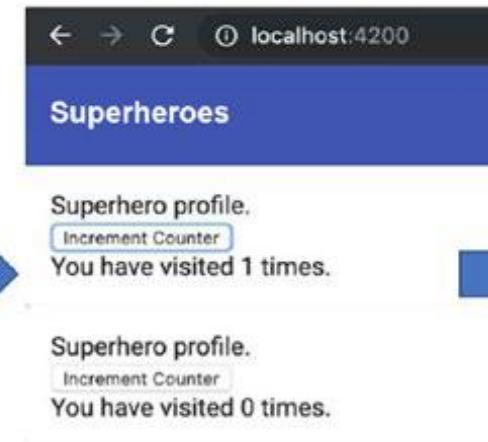
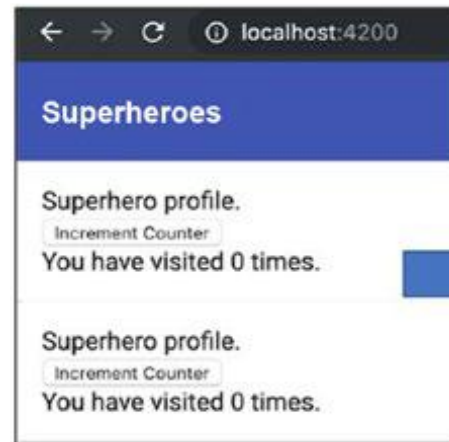
7 - Angular: Services and Dependency Injection

Voorbeeld HitCounter

- module level injection



- component level injection



— Angular for Material Design

7 - Angular: Services and Dependency Injection

Exercise

- dataservice voor de dinosaur data
 - provided op module niveau
 - asynchronously met promise of observable
- logger service die dat de data afbeeldt in de console
 - provided op root niveau

— Angular for Material Design

8 - Material Design: User Input

- In dit hoofdstuk user input met text fields, single or multiselect drop-down, date controls, ...
- Material input elements

Superhero Name

Enter name

Without keyboard focus

Superhero Name

A name to be remembered

Enter name

With keyboard focus

Floating label

Placeholder

Hint Text

— Angular for Material Design

8 - Material Design: User Input

- Material Design Form Field
 - text field
 - text area
 - drop-down
 - chip input (selecting multiple items)
- mat-form-field: appearance, floating labels, hint text and place holders
- importeer in SuperheroesMaterialDesignModule: MatFormsModule en MatInputModule
- maak component CreateSuperhero

— Angular for Material Design

8 - Material Design: User Input

mat-form-field appearance

Superhero Name
A name to be remembered
Enter name

Appearance: Legacy

Superhero Name
A name to be remembered
Enter name

Appearance: Fill

Superhero Name
A name to be remembered
Enter name

Appearance: Standard

Superhero Name
A name to be remembered
Enter name

Appearance: Outline

— Angular for Material Design

8 - Material Design: User Input

Text Field within mat-form-field

- Listing 8-6

```
<div>
  <mat-form-field appearance="standard">
    <input type="text" matInput placeholder="A name to be remembered" />
  </mat-form-field>
</div>
```

A name to be remembere

— Angular for Material Design

8 - Material Design: User Input

Floating Label (auto => floats above when user focuses)

- Listing 8-7

```
<mat-form-field floatLabel="auto" appearance="standard">  
  <mat-label>Superhero Name</mat-label>  
  <input type="text" required matInput placeholder="A name to be remembered" />  
</mat-form-field>
```

Superhero Name *

A name to be remembere

— Angular for Material Design

8 - Material Design: User Input

Required Field

- Listing 8-8

```
<div>
  <!-- Appearance: Outline -->
  <mat-form-field appearance="outline">
    <mat-label>Superhero Name</mat-label>
    <input type="text" matInput required placeholder="A name to be remembered" />
    <mat-hint>Enter name</mat-hint>
  </mat-form-field>
</div>
```

Superhero Name *

A name to be remembered

Enter name

— Angular for Material Design

8 - Material Design: User Input

Input Types with matInput

- Date and time-related types: date, month, time, week, datetime-local
- User data types: password, email, tel, url
- Other frequently used types: number, search, color
- Not supported types
 - checkbox => wel met MatCheckboxModule
 - file
 - hidden
 - image
 - radio
 - range
 - reset
 - submit

— Angular for Material Design

8 - Material Design: User Input

Tekst Area Within mat-form-field

- Listing 8-11

```
<mat-form-field appearance="outline">  
  <mat-label>Powers</mat-label>  
  <textarea rows="6" matInput></textarea>  
  <mat-hint>Explain superhero powers</mat-hint>  
</mat-form-field>
```

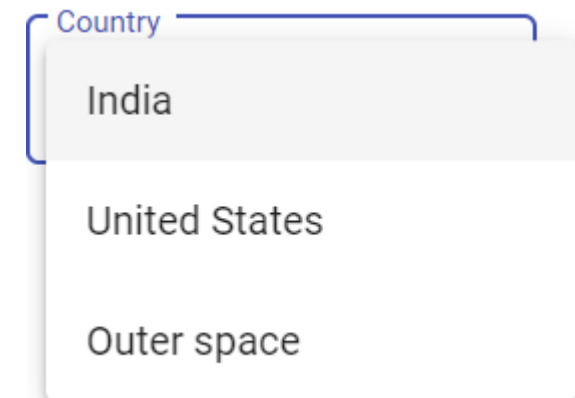
— Angular for Material Design

8 - Material Design: User Input

Material Drop-downs

- voeg MatSelectModule toe:
import { MatSelectModule } from '@angular/material/select';
- Listing 8-14

```
<div>  
  <!-- Appearance: Outline -->  
  <mat-form-field appearance="outline">  
    <mat-label>Country</mat-label>  
    <mat-select>  
      <mat-option>India</mat-option>  
      <mat-option>United States</mat-option>  
      <mat-option>Outer space</mat-option>  
    </mat-select>  
  </mat-form-field>  
</div>
```



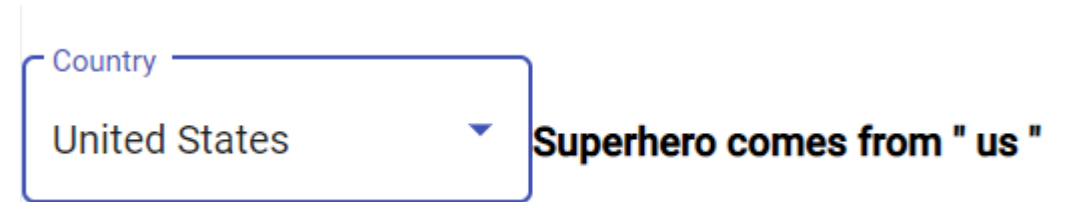
— Angular for Material Design

8 - Material Design: User Input

Material Drop-downs

- Listing 8-15 - value of mat-select

```
<div>
  <!-- Appearance: Outline -->
  <mat-form-field appearance="outline">
    <mat-label>Country</mat-label>
    <mat-select [(value)]="country">
      <mat-option>None</mat-option>
      <mat-option value="in">India</mat-option>
      <mat-option value="us">United States</mat-option>
      <mat-option value="os">Outer Space</mat-option>
    </mat-select>
  </mat-form-field>
  <strong>Superhero comes from " {{country}} " </strong>
</div>
```



— Angular for Material Design

8 - Material Design: User Input

Material Drop-downs

- Listing 8-16 - Categorize
- Listing 8-17 - Multiple select

— Angular for Material Design

8 - Material Design: User Input

Material Design: Chips

- interface om één of meerdere items te selecteren
- importeer de chips module
import { MatChipsModule } from '@angular/material/chips';

- Basic Chip List

```
<div>  
  <mat-form-field appearance="standard">  
    <mat-label>Likes to eat</mat-label>  
    <mat-chip-list #favFood>  
      <mat-chip> Laddu </mat-chip>  
      <mat-chip> Cheeseburger </mat-chip>  
    </mat-chip-list>  
    <input [matChipInputFor]="favFood" />  
  </mat-form-field>  
</div>
```

Likes to eat

Laddu

Cheeseburger

— Angular for Material Design

8 - Material Design: User Input

Material Design: Chips

- Listing 8-20: Add items Chip List

```
<mat-form-field appearance="standard">
  <mat-label>Likes to eat</mat-label>
  <mat-chip-list #favFood>
    <mat-chip *ngFor="let item of items">
      {{item}}
    </mat-chip>
  </mat-chip-list>
  <input [matChipInputFor]="favFood"
    (matChipInputTokenEnd)="addItem($event)"
    matChipInputAddOnBlur />
</mat-form-field>
```

```
items: Array<string> = ["Laddu", "Cheeseburger",
"Pizza"];
addItem(event: any) {
  this.items.push(event.value);
}
```

Likes to eat

Laddu

Cheeseburger

Pizza

Lasagne

Spaghetti

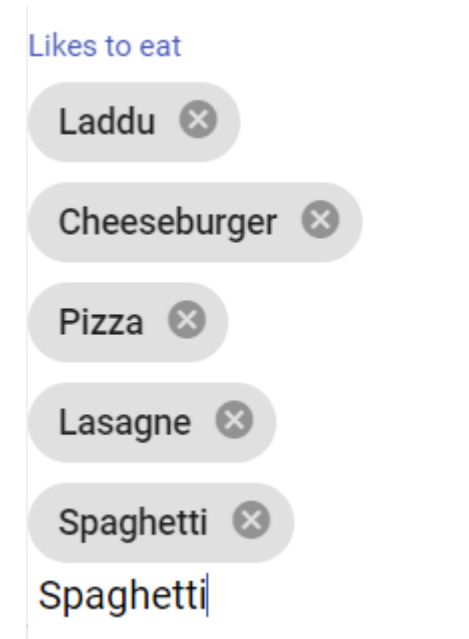
— Angular for Material Design

8 - Material Design: User Input

Material Design: Chips

- Listing 8-22: Remove items from a Chip List

```
removeHandler(item: string) {  
    this.items.splice(this.items.indexOf  
(item), 1);  
}  
  
<mat-form-field appearance="standard">  
    <mat-label>Likes to eat</mat-label>  
    <mat-chip-list #favFood>  
        <mat-chip *ngFor="let item of items" (removed)="removeHandler(item)">  
            {{item}}  
            <mat-icon matChipRemove>cancel</mat-icon>  
        </mat-chip>  
    </mat-chip-list>  
    <input [matChipInputFor]="favFood" (matChipInputTokenEnd)="addItem($event)" matChipInputAddOnBlur  
</mat-form-field>
```



— Angular for Material Design

8 - Material Design: User Input

Exercise

- Bouw een dinosaur form
 - dinosaur naam
 - dinosaur family, met drop-down
 - Timeframe: text field met Mil-yrs-ago als postfix
 - Dinosaurus descriptie: text area: min 6 lijnen, max 20
 - Continents: chip list

— Angular for Material Design

9 - Angular: Building Forms

- data is input through a form - validated and saved in the backend
- Angular framework has two options for creating forms
 - Template-driven forms: simple
 - Reactive forms: Model driven -> for complex forms

— Angular for Material Design

9 - Angular: Building Forms

- Template-Driven Forms
 - easy to build
 - asynchronous
 - data is mutable (change of the form -> change of the model object/Typescript class field)
- voeg FormsModule toe (is vroeger reeds gedaan voor ngModel)
- maak een superhero Model (vroeger reeds gedaan voor dataservices)
- Listing 9-3: voeg het superhero Model toe aan CreateSuperhero
- Listing 9-4: voeg een form toe: `<form #superheroForm="ngForm">`
 - #superheroForm: view variable - kan dan later in de view gebruikt worden
- Listing 9-5: voeg de formfields toe en gebruik databinding met [(ngModel)]
`<input type="text" name="name" [(ngModel)]="superhero.name" matInput required placeholder="A name to be remembered" />`
- Listing 9-6: two way databinding van mat-select is mogelijk met value variabele
`<mat-select placeholder="select country" [(value)]="superhero.country">`

— Angular for Material Design

9 - Angular: Building Forms

Voeg Getter toe aan CreateSuperhero

```
get model() {  
  return JSON.stringify(this.superhero);  
}
```

```
<strong>  
  {{model}}  
</strong>
```

of {superhero | json} in de HTML template

— Angular for Material Design

9 - Angular: Building Forms

Form validation and errors

- listing 9-9: voeg attributes toe
 - required
 - minlength en maxlength
 - email

Email

How do I contact the superhero for help?

— Angular for Material Design

9 - Angular: Building Forms

Using the State Field in Form Controls

- touched/untouched
- dirty/pristine
- valid/invalid

```
<div>
  <mat-form-field appearance="outline">
    <mat-label>Email</mat-label>
    <input type="text" name="email" email [(ngModel)]="superhero.email" matInput #email="ngModel"/>
    <mat-hint>How do I contact the superhero for help?</mat-hint>
  </mat-form-field>
</div>
<!-- Show error message to the user -->
<div class="error-message" *ngIf="!email.valid && email.touched">
  Please enter a valid email address.
</div>
```

— Angular for Material Design

9 - Angular: Building Forms

State of the Form

```
<form #superheroForm="ngForm">
  <strong>
    Form Status, Valid- {{superheroForm.valid}}.
    Pristine- {{superheroForm.pristine}}.
    Touched- {{superheroForm.touched}}
  </strong>
```

— Angular for Material Design

9 - Angular: Building Forms

Submit/save on an invalid form

```
<div>  
  <button mat-stroked-button> Cancel </button>  
  <button mat-stroked-button [disabled]="!superheroForm.valid"> Save  
  </button>  
</div>
```

in superheroes-material-design.module.ts:
import toevoegen van "MatButtonModule"

— Angular for Material Design

9 - Angular: Building Forms

Reactive forms

- Model driven: een data object bepaalt de form representatie
- Immutable: iedere verandering => volledig nieuwe state (dus geen verandering aan de bestaande state)
- Reactive forms laten unit testing toe - aanstuurbaar vanuit component

in superheroes-material-design.module.ts:
import toevoegen van "ReactiveFormsModule"

ng g component superheroes-material-design/create-superhero-reactive-form

— Angular for Material Design

9 - Angular: Building Forms

Create a Form Control with Form Builder

- CreateSuperheroReactiveFormComponent.ts:

```
// Create an instance of FormControl using the FormBuilder  
name: FormControl = this.fb.control("");
```

```
// Inject FormBuilder  
constructor(private fb: FormBuilder) { }
```

- create-superhero-reactive-form.component.html:

```
<mat-form-field appearance="outline">  
  <mat-label>Superhero Name</mat-label>  
  <input type="text" name="name" [formControl]="name" matInput  
    placeholder="A name to be remembered" />  
  <mat-hint>Enter name</mat-hint>  
</mat-form-field>
```

— Angular for Material Design

9 - Angular: Building Forms

Capture Changes to the Form Control

- gebruikersacties => stroom van interacties => observable met valueChanges

```
ngOnInit(): void {  
  this.name.valueChanges.subscribe( result => console.log(result));  
}
```

- change (na enter)

```
<input type="text" name="name" [formControl]="name" matInput  
placeholder="A name to be remembered" (change)="changeHandler()"/>
```

— Angular for Material Design

9 - Angular: Building Forms

Form control instellen

- mimic gedrag van instellen na API call (waarde met vertraging ter beschikking)

```
ngOnInit(): void {  
  this.name.valueChanges.subscribe( result => console.log(result));  
  setTimeout( () => this.name.setValue("Value set on timeout"), 1000);  
}
```

1	create-superhero-rea...orm.component.ts:18
12	create-superhero-rea...orm.component.ts:18
Log the snapshot at a point in time, 12	create-superhero-rea...orm.component.ts:23
Value set on timeout	create-superhero-rea...orm.component.ts:18
>	

— Angular for Material Design

9 - Angular: Building Forms

Create a Form Group with Form Builder
listing 9-23 en 9-24

```
// Class variable representing FormGroup.
superheroFormGroup: FormGroup;

// Inject FormBuilder
constructor(private fb: FormBuilder) {
  // Create FormGroup object with FormBuilder.
  this.superheroFormGroup = this.fb.group({
    name: '',
    email: '',
    details: '',
    powers: '',
    country: ''
  });
}
```

```
<form #superheroForm="ngForm"
  [formGroup]="superheroFormGroup">
  <input type="text" name="name"
    FormControlName="name" ...
```

— Angular for Material Design

9 - Angular: Building Forms

Read Form Group Values as a Stream
listing 9-26

```
this.superheroFormGroup
  .valueChanges
  .subscribe(item =>
    console.log("Stream as form changes, ",
item as Superhero));
```

[WDS] Live Reloading enabled.

Value set on timeout

Stream as form changes,

▶ {name: '5', email: '', details: '', powers: '', country: ''}

Stream as form changes,

▼ {name: '56', email: '', details: '', powers: '', country: ''} ⓘ

country: ""

details: ""

email: ""

name: "56"

powers: ""

▶ [[Prototype]]: Object

Stream as form changes,

▶ {name: '565', email: '', details: '', powers: '', country: ''}

Stream as form changes,

▶ {name: '5654', email: '', details: '', powers: '', country: ''}

Stream as form changes,

▶ {name: '56545', email: '', details: '', powers: '', country: ''}

Stream as form changes,

▶ {name: '565454', email: '', details: '', powers: '', country: ''}

>

— Angular for Material Design

9 - Angular: Building Forms

Read Form Group Values as a Snapshot
listing 9-28 en 9-29

```
<form #superheroForm="ngForm" [formGroup]="superheroFormGroup"  
(submit)="submitHandler()">
```

— Angular for Material Design

9 - Angular: Building Forms

Form Validation

listing 9-30, ..., 9.34

```
// Create an instance of FormControl using the FormBuilder
name: FormControl = this.fb.control("", [Validators.required]);

// Inject FormBuilder
constructor(private fb: FormBuilder) {
  // Create FormGroup object with FormBuilder.
  this.superheroFormGroup = this.fb.group({
    name: ['', Validators.required],
    email: ['', [Validators.required, Validators.email]],
    details: ['', [Validators.minLength(5), Validators.maxLength(100)]],
```

— Angular for Material Design

9 - Angular: Building Forms

Valid/Invalid

listing 9-35, ..., 9.36

```
get email(){  
    return this.superheroFormGroup  
        .get("email");  
}
```

```
<mat-icon *ngIf="email?.valid" matSuffix >check_circle</mat-icon>  
<mat-hint>How do I contact the superhero for help?</mat-hint>  
</mat-form-field>  
</div>  
<div *ngIf="email?.invalid" class="error-message">  
    Invalid email. Use YourEMail@DomainName.com  
</div>
```

— Angular for Material Design

9 - Angular: Building Forms

- touched/untouched

listing 9-38

```
<div *ngIf="email?.touched && email?.invalid" class="error-message">
  Invalid email. Use YourEmail@DomainName.com
</div>
```

- pristine/dirty

```
<strong>
  Pristine: {{ email?.pristine }}. Dirty: {{ email?.dirty }}
</strong>
```

- error vlag

```
<div *ngIf="email?.errors?.required">
  *Email is a required field
</div>
```



— Angular for Material Design

10 - Material Design: Additional Form Fields

extra controls:

- date picker
- slider
- toggle switches
- checkboxes



— Angular for Material Design

10 - Material Design: Additional Form Fields

- Date picker
 - MatDatepickerModule
 - MatNativeDateModule
- moment
 - npm install moment
 - npm install @angular/material-moment-adapter
- Listing 10-3, 10-5

```
<input matInput formControlName="dob" [matDatepicker]="dateOfBirth" />
<!--mat-datepicker #dateOfBirth [opened]="true"></mat-datepicker-->
<mat-datepicker [opened]="true" startView="multi-year" #dateOfBirth ></mat-datepicker>
```

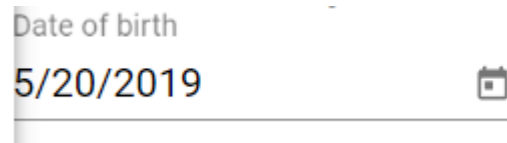
```
console.log("Selected date value ",
moment(this.superheroFormGroup.value.dob).toDate());
```

— Angular for Material Design

10 - Material Design: Additional Form Fields

Toggle Date picker

- Listing 10-9



```
<div>
  <mat-form-field>
    <mat-label>Date of birth</mat-label>
    <input matInput formControlName="dob" [matDatepicker]="dateOfBirth" />
    <!--mat-datepicker #dateOfBirth [opened]="true"></mat-datepicker-->
    <mat-datepicker-toggle matSuffix [for]="dateOfBirth"></mat-datepicker-toggle>
    <mat-datepicker startView="multi-year" #dateOfBirth></mat-datepicker>
  </mat-form-field>
</div>
```



— Angular for Material Design

10 - Material Design: Additional Form Fields

Date Picker

- custom icon
- filter dates: listing 10-11
- custom filter: listing 10-14

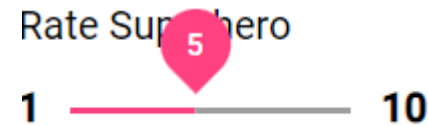


— Angular for Material Design

10 - Material Design: Additional Form Fields

Slider, Listing 10-17,10-18,10-19

```
<div>
  <div>
    Rate Superhero
  </div>
  <div>
    <strong> {{ minRating }} </strong>
    <mat-slider formControlName="rating" [max]="maxRating"
[min]="minRating" step="1" thumbLabel>
    </mat-slider>
    <strong> {{ maxRating }} </strong>
  </div>
</div>
```



— Angular for Material Design

10 - Material Design: Additional Form Fields

Toggle Switch and Check Box

- voeg toe: MatSlideToggleModule en MatCheckboxModule
- Listing 10-24, 10-25

Add a cape to the hero? 

```
<mat-slide-toggle checked="true" (change)="onToggleChange($event)" labelPosition="before"
disableRipple="true">
```

```
Add a cape to the hero? </mat-slide-toggle>
```

```
onToggleChange(event: MatSlideToggleChange) {
  console.log(event.checked);
}
```

— Angular for Material Design

10 - Material Design: Additional Form Fields

Toggle Switch and Check Box

- Listing 10-26, 10-27

```
let j = this.cities.map( i => new FormControl(i.selectedByDefault));

this.superheroFormGroup = new FormGroup( {
  name: new FormControl('', Validators.required),
  email: new FormControl('', [Validators.required, Validators.email]),
  details: new FormControl('', [Validators.minLength(5), Validators.maxLength(100)]),
  powers: new FormControl(''),
  country: new FormControl(''),
  dob: new FormControl(new Date('05/20/2019')),
  rating: new FormControl(2),
  jurisdiction: new FormArray(j)
});
```

— Angular for Material Design

10 - Material Design: Additional Form Fields

Toggle Switch and Check Box

- Listing 10-26, 10-27

```
cities = [ {  
  name: "New York",  
  selectedByDefault: true  
},{  
  name: "Boston",  
  selectedByDefault: false  
},{  
  name: "Hyderabad",  
  selectedByDefault: false  
}, {  
  name: "Bengaluru",  
  selectedByDefault: true  
}];
```

```
<div formArrayName="jurisdiction">  
  <div *ngFor="let city of cities; let i=index;" >  
    <mat-slide-toggle [formControlName]="i">  
      {{cities[i].name}}  
    </mat-slide-toggle>  
  </div>  
</div>
```



— Angular for Material Design

10 - Material Design: Additional Form Fields

Toggle Switch and Check Box

- Listing 10-26, 10-27

```
this.superheroFormGroup
  .valueChanges
  .subscribe(data => {
    data.jurisdiction.map((value:boolean, it: number)
      => console.log(this.cities[it].name, value));
  });
```

Select Superhero's jurisdiction

- ☒ New York
- ☒ Boston
- ☒ Hyderabad
- ☒ Bengaluru

```
Stream as form changes,
  ▶ {name: '', email: '', details: '', powers: '', country: '', ...}
New York true
Boston true
Hyderabad true
Bengaluru true
```

— Angular for Material Design

10 - Material Design: Additional Form Fields

In the dinosaurs form, provide options to select the continents that a given dinosaur was found in. Create the form with the following toggle button options. Selecting at least one continent is mandatory.

- North America
- South America
- Greater India
- Europe
- Arabia
- Laurasia
- Gondwanaland

Provide a date-of-discovery option. Do not allow the date to be prior to 1900. The date cannot be later than today (the current system date).

— Angular for Material Design

11 • Angular: Routing

- Meeste Angular Apps zijn SPA (single-page applications)
- Navigeren tussen pagina's
 - gedeelte van de pagina wordt herladen
 - header, footer, navigation sidebar, ... worden enkel indien nodig herladen
- Waarom routing?
 - gebruikers navigeren tussen de verschillende views en componenten
 - browsers gebruiken URL's en history data voor forward/backward functionaliteit
 - browsers gebruiken URL's om pagina's te bookmarken
- Routing => URLs koppelen aan een view

— Angular for Material Design

11 • Angular: Routing

gestart met template inclusief routing

- @angular/router wordt geïnstalleerd
- AppRoutingModuleModule wordt gemaakt:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

const routes: Routes = [];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

— Angular for Material Design

11 • Angular: Routing

AppModule importeert de routing module:

```
import { AppRoutingModule } from './app-routing.module';
```

```
@NgModule({  
  declarations: [  
    AppComponent  
  ],  
  imports: [  
    BrowserModule,  
    AppRoutingModule,  
    SuperheroesMaterialDesignModule  
  ],  
  providers: [],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

— Angular for Material Design

11 • Angular: Routing

Route Configuration

- URL gekoppeld aan een View
 - path: URL
 - component: Angular Component

```
const routes: Routes = [  
  {  
    path: "heroes",  
    component: SuperheroListComponent  
  },  
  {  
    path: "create-hero",  
    component: CreateSuperheroReactiveFormComponent  
  }  
];
```

— Angular for Material Design

11 • Angular: Routing

Router Outlet: placeholder waar de te tonen component wordt in getoond
app.component.html:

```
<superhero-toolbar></superhero-toolbar>
```

```
<router-outlet>  
</router-outlet>
```

— Angular for Material Design

11 • Angular: Routing

Link to route

- app.component.html:
routerlink => App wordt niet volledig opnieuw geladen van server, enkel "navigeren"
- superhero-toolbar.component.html:

```
<mat-toolbar color="primary">Superheroes  
  <ng-content></ng-content>  
</mat-toolbar>
```

```
<superhero-toolbar>  
  {{title}}  
  <a routerLink="/create-hero">  
    Create Hero  
  </a>  
</superhero-toolbar>  
  
<router-outlet>  
</router-outlet>
```

— Angular for Material Design

11 • Angular: Routing

Route parameters
Listing 11-11 en 11-12

```
const routes: Routes = [  
  // Removed other routes for brevity  
  {  
    path: "hero/:superheroName",  
    component: SuperheroComponent  
  }  
];
```

```
constructor(private route: ActivatedRoute)
```

```
  ngOnInit() {  
    const heroNameParam = this.route.snapshot.params.heroName || "";  
    // find hero in a list of heroes.  
    this.hero = this.heroes.find( hero => hero.name.toLowerCase() ===  
      heroNameParam.toLowerCase());  
  }
```

— Angular for Material Design

11 • Angular: Routing

- Using an Observable to Read Route Param
Listing 11-14
- Link a Route with Parameters

```
<div *ngFor="let hero of heroes">  
  <a [routerLink]="['/hero', hero.name]">  
    <mat-card>  
      <!-- removed code for brevity -->  
    </a>  
  </div>
```

— Angular for Material Design

11 • Angular: Routing

- Default route, Listing 11-23

```
const routes: Routes = [  
  // Removed other routes for brevity  
  {  
    path: "",  
    redirectTo: "/heroes",  
  },  
  {  
    path: "**",  
    redirectTo: "/heroes"  
  }  
];
```
- Undefined route

— Angular for Material Design

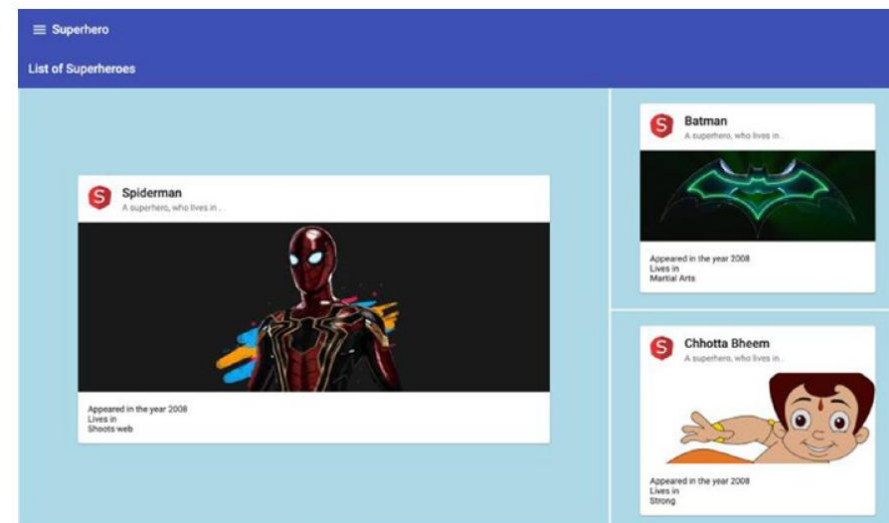
Oefening Dinosaurus route

- Maak een component DinosaurList, met een lijst van dinosaurussen. Deze component moet geopend worden met de default route.
- Voorzie een route naar de individuele dinosaurussen: /dinosaur/:dinosaurName.

— Angular for Material Design

hoofdstuk 12,13,14,15

- 12. Material Design: Navigation
 - Toolbar
 - Integration with Angular Router: `routerLink="/heroes"` => niet hele pagina opnieuw laden ...
 - Sidenav
- 13. Material Design: Layout
 - Cards, Tabs, Expansion Panel
- 14. Material Design: Using Lists
 - List, Grid List
- 15. Material Design: Alerts and Dialogs



— Angular for Material Design

16 - Angular HTTP Client

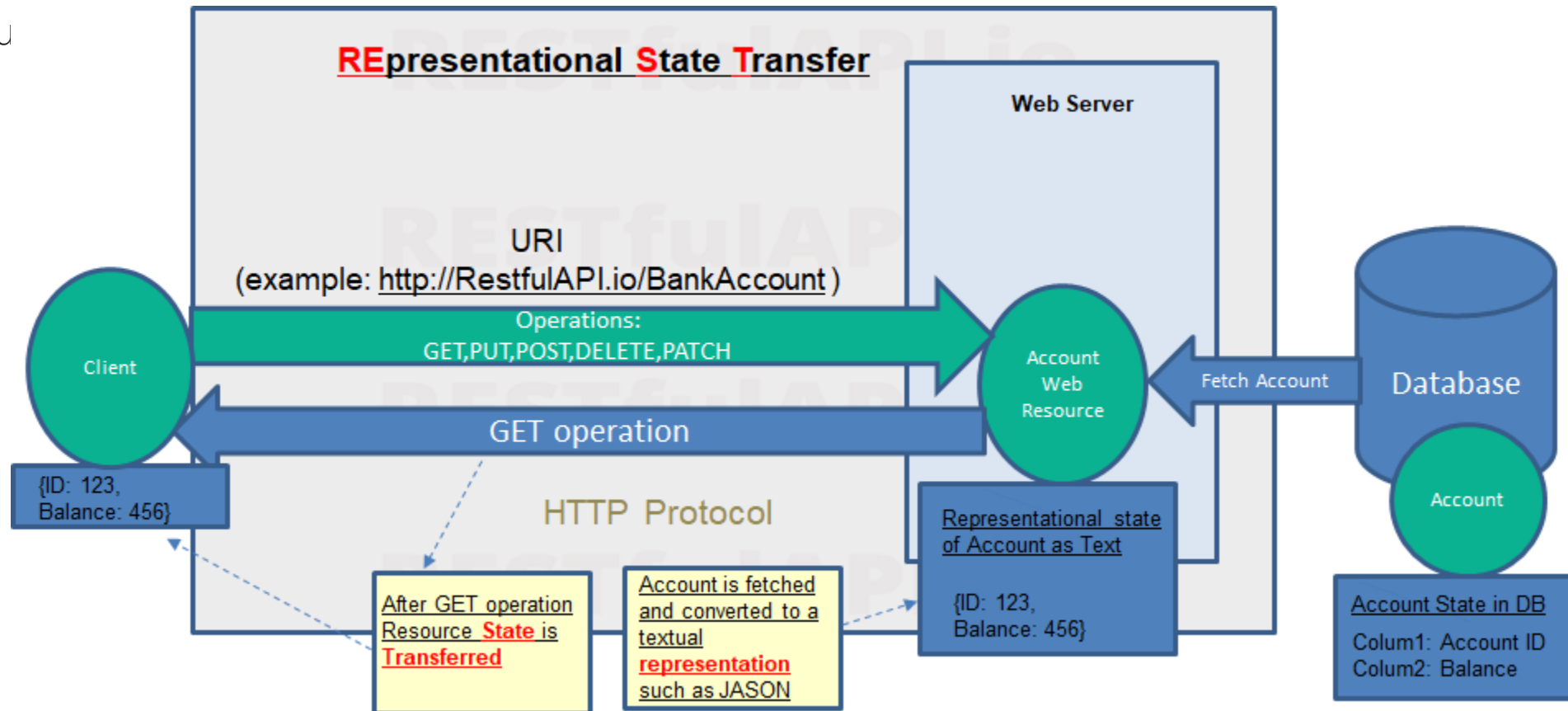
HTTP Client

- For AJAX calls (XMLHttpRequest (XHR) over HTTP)
- Met JSON
- RESTful – GET, POST, PUT en DELETE

— Angular for Material Design

16 – Angular HTTP Client

Restfu



— Angular for Material Design

16 – Angular HTTP Client

Restful

GET – gets the current state of the resource. Idempotent.

PUT – works on a single resource and for creation or changes of its state. Idempotent

DELETE – Deletes the specified object. Idempotent.

POST – works on a collection. Creates or updates a collection.

PATCH – updates a collection

Developers are often confused on PUT vs POST. As a general practice take advantage of idempotency of PUT and use it for creation and POST for update.

— Angular for Material Design

16 – Angular HTTP Client

	HTTP Method	Idempotency	Reason
Restful	GET	Idempotent	GET used for retrieving information, multiple retrieval does not change change state of the resource. GET is Idempotent.
	POST	Not Idempotent	POST could be used to create a resource. for this reason resource id is is not mentioned in URI. Calling a POST method multiple times will create resources each time. So, POST is not Idempotent
	DELETE	Idempotent	DELETE is used to delete a specific resource. Multiple attempt to DELETE yields the same result. DELETE is Idempotent.
	PUT	Idempotent	PUT is used for update or replace the resource mentioned in URI. For URI. For this reasons, a resource id is generally mentioned in the URI. URI. Attempt to update the same resource with same values multiple times does not change the state of that resource. PUT is Idempotent.
	PATCH	Not Idempotent	PATCH is used to update a portion of the resource. PATCH is not Idempotent

— Angular for Material Design

16 – Angular HTTP Client

- Module “app-http-calls” aanmaken:
ng g module app-http-calls
- Generate a Service for Service Calls
ng g service app-http-calls/superhero-data
- Importeer HttpClientModule in AppHttpCalls
- Importeer AppHttpCalls in AppModule

— Angular for Material Design

16 – Angular HTTP Client

- Mock data
 - npm install -g json-server
 - json-server data.json
angular-for-material-design/data.json at master · hans-naert/angular-for-material-design (github.com)
- SuperheroDataService (Listing 16-7)
- Add component superhero-grid-list
ng generate component superheroes-material-design/superhero-grid-list
- Add route for component

```
path: "hero-grid-list",  
component: SuperheroGridListComponent
```

— Angular for Material Design

16 – Angular HTTP Client

in superhero-grid-list:

```
constructor(private heroService: SuperheroDataService) { }  
  
ngOnInit(): void {  
    this.heroService.getHeroes();  
}
```

http://localhost:4200/hero-grid-list

[superhero-data.service.ts:15](#)

```
▼ (6) [{...}, {...}, {...}, {...}, {...}, {...}] ⓘ  
  ► 0: {name: 'Batman', email: 'batman@angularsample.com', details: '...'  
  ► 1: {name: 'Chhotta Bheem', email: 'ChhottaBheem@angularsample.com...'  
  ► 2: {name: 'Spiderman', email: 'spiderman@angularsample.com', deta...  
  ► 3: {name: 'Antman', email: 'antman@angularsample.com', details: '...'  
  ► 4: {name: 'Thor', email: 'thor@angularsample.com', details: 'God ...'  
  ► 5: {name: 'Wonder Woman', email: 'ww@angularsample.com', details:...'  
    length: 6  
  ► [[Prototype]]: Array(0)
```

— Angular for Material Design

16 – Angular HTTP Client

in SuperheroDataService - listing 16.8 (with map)

```
getHeroes() {  
  let heroes: Observable<any> = this.httpClient.get(`${URL_PREFIX}/heroes`);  
  heroes.subscribe(  
    (data) => {  
      console.log(data.map((hero: any) => ({  
        "name": hero.name || "",  
        "email": hero.email || "",  
        "details": hero.details || "",  
        "country": hero.country || "",  
        "specialPowers": hero.specialPowers || [],  
        "cardImage": hero.cardImage || "/defaultImage.jpg",  
        "favFood": hero.favFood || []  
      })))  
    },  
    () => ({}))/** Error handling callback */,  
    () => ({}))/** Done with the observable */,  
  );  
}
```

— Angular for Material Design

16 – Angular HTTP Client

in SuperheroDataService - listing 16.8 (with map)
use subscribe with object

```
heroes.subscribe(  
  {next :(data) => {  
    ...  
  },  
  error: () => ({})** Error handling callback **,  
  complete: () => ({})** Done with the observable **,  
})
```

— Angular for Material Design

16 – Angular HTTP Client

Explicitly Type Response Object - listing 16.8 (with map)

```
13  getHeroes() {
14  |  let heroes: Observable<Array<Superhero>> = this.httpClient.get<Array<Superhero>>(`${URL_PREFIX}/heroes`)
15  |  heroes.subscribe(
16  |  |  {next :(data) => {
17  |  |  |  console.log(data.map((hero) => ({
18  |  |  |  |  "name": hero.name || "",
19  |  |  |  |  "email": hero. cardImage (property) Superhero.cardImage: string
20  |  |  |  |  "details": he country
21  |  |  |  |  "country": he details
22  |  |  |  |  "specialPower email
23  |  |  |  |  "cardImage": favFood
24  |  |  |  |  "favFood": he isExpanded?
25  |  |  |  |  name
26  |  |  |  |  specialPowers
27  |  |  |  |  })))
28  |  |  },
29  |  |  error: () => ({})) /** Error handling callback */
```

— Angular for Material Design

16 – Angular HTTP Client

- Return Observable from HTTP Client to the Calling Function - listing 16-10
- Results from the Angular Service Assigned to a Class Variable - listing 16-11
- Results from the Angular Service Assigned to a Class Variable - 16-12

```
<p>superhero-grid-list works!</p>
<mat-grid-list cols=1 rowHeight="100px">
  <mat-grid-tile *ngFor="let hero of heroes; let i=index">
    <mat-card>
      {{hero.name}}
    </mat-card>
  </mat-grid-tile>
</mat-grid-list>
```

— Angular for Material Design

16 – Angular HTTP Client

- assigning Observable to Class variable

```
heroes: Observable<Array<Superhero>> | undefined;
```

```
ngOnInit(): void {  
    this.heroes=this.heroService.getHeroes();  
}
```

- using observable in template

```
<mat-grid-list cols=1 rowHeight="100px">  
    <mat-grid-tile *ngFor="let hero of heroes | async; let i=index">
```